GALACTIC PRIME TIME

Full Developer Handoff

Lights. Camera. Action.

Project	Galactic Prime Time — 2.5D tactical RPG adapted from a custom TTRPG system
Engine	Godot 4 · GDScript
Database	SQLite (static data) + JSON files (dynamic save state)
Architecture	MVC — headless simulation, GameController singleton, Godot scene tree
Jira	galacticprimetime.atlassian.net · Project key: KAN
Start at	KAN-8 — Design SQLite schema. Do not begin any other ticket until complete.
This doc contains	GDD · ERD · DFD · Architecture diagram · Full ticket order · Codex prompt

Contents

1. Project Vision & Pillars
2. Architecture Overview
3. Data Layer — SQLite Schema (ERD)
4. Data Flow Diagram (DFD)
5. System Architecture Diagram
6. Combat System Rules
7. Key Systems — Skills, Tags, Exposure, Ascension
8. Campaign Structure & Content Schemas
9. Godot Folder Structure
10. Jira Ticket Order — Full Build Sequence
11. Codex System Prompt & Per-Ticket Instructions

1. Project Vision & Pillars

Galactic Prime Time is a 2.5D tactical RPG. Abducted humans compete in a dungeon crawl broadcast as intergalactic reality TV to an alien audience. The player builds a party and fights through three dungeon floors while the audience watches, sets challenges, and funds contestants. The core fantasy is the underdog arc — nobody becomes somebody under the worst possible conditions.

Core Pillars

- Spectacle over safety — passive play is structurally discouraged by the system.
- Identity through Tags — repeated behavior earns labels that reshape how the world responds.
- The Audience as an active mechanic — Viewers, Followers, and Patrons react to everything.
- Timeline combat — shared 10-Moment Clock, no turns, all actions overlap.
- Nobody to somebody — early difficulty is intentional. Players must struggle to feel meaningful growth.

Tone References

Dungeon Crawler Carl · The Running Man · LitRPG genre · MMO design sensibility · Dark corporate satire

Campaign Structure

Tutorial	Boss fight introduction. Unlocks The Lounge on completion.
Floor 1	Green forest. Three routes (Easy/Medium/Hard). Players pick one, others play out offscreen.
Floor 2	Great desert. Same routes, 70 years later. Floor 1 consequences visible in world state.
Floor 3	Grand capital city. 100 years after Floor 2. Routes converge toward finale.

2. Architecture Overview

The game is built in three strictly separated layers. This separation is not optional — it enables checkpoint serialization, headless testing, and clean content pipelines.

Model — Headless Simulation

Pure GDScript classes with zero Godot scene dependencies. Contains: Clock event queue, CombatantState, ActionResolver, ForcedActionSystem, ConditionEngine, ResistanceSystem, ExposureEngine. This layer runs, is tested, and is serialized without launching Godot.

Controller — GameController Singleton

The only layer allowed to touch both the simulation and the scene tree. Translates player input into simulation commands. Receives simulation events and routes them to correct scene nodes. Owns the SaveManager and DataAccessLayer. All SQLite queries go exclusively through the DAL — never directly from simulation or scenes.

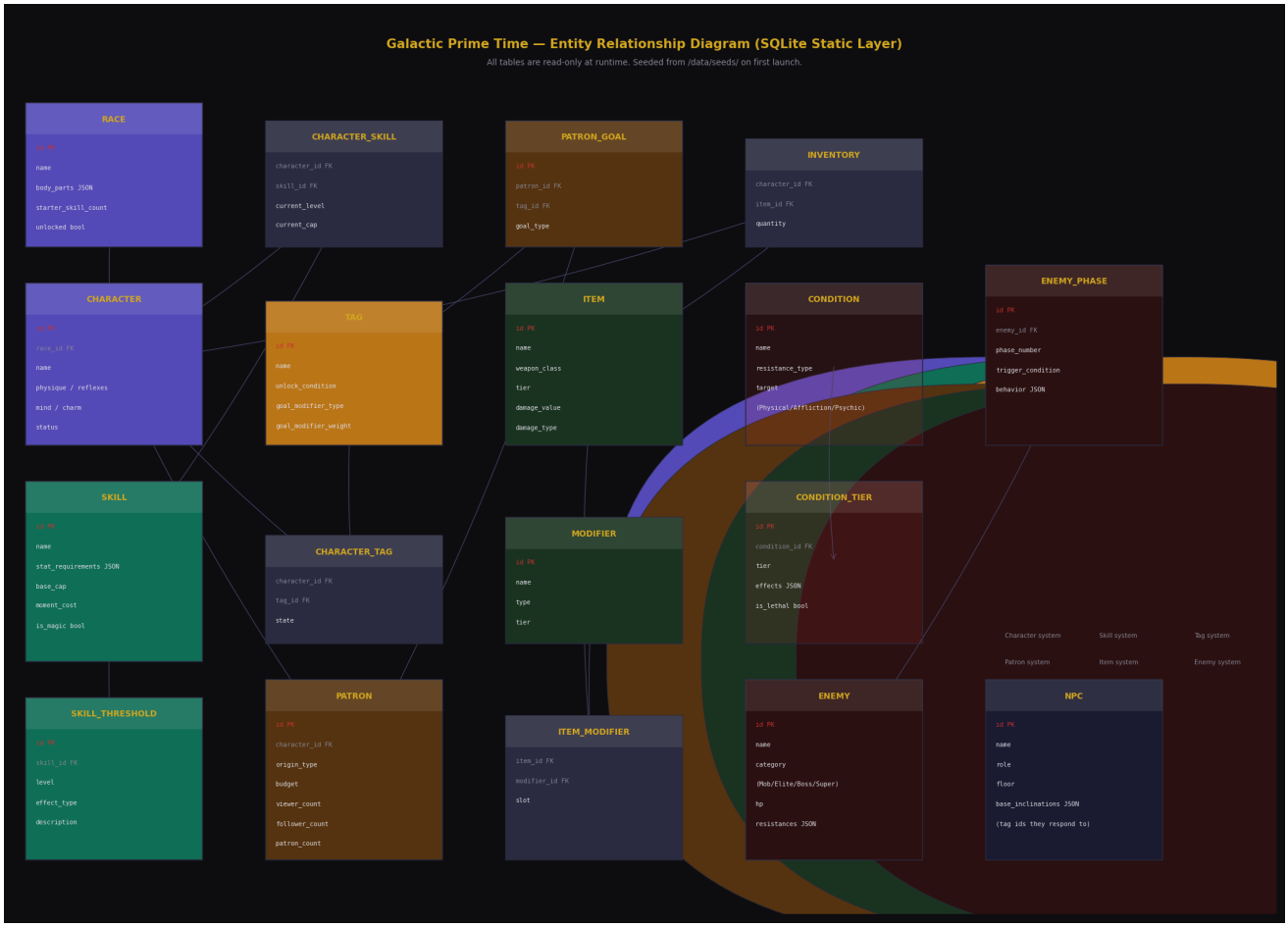
View — Godot Scene Tree

Pure presentation. Scenes listen to GameController signals and render what they are told. Contains: HexGridRenderer, CombatantSprites, HUD, PopupFramework, BroadcastLayer, AudienceUI. No scene ever reads SQLite directly or touches simulation state.

CRITICAL RULE: No game logic in scenes. No SQLite queries outside the DAL. No simulation logic in the Controller.

3. Data Layer — SQLite Schema (ERD)

All tables listed below are read-only at runtime. They are seeded on first launch from JSON/CSV files in /data/seeds/. The DataAccessLayer is the only class that queries these tables.



Entity Relationship Diagram — Galactic Prime Time SQLite static layer

Table Notes

- races — body_parts stored as JSON array (e.g. ["Head","Torso","Arm","Arm","Leg","Leg"]). unlocked bool controls whether race is available at character creation.
- skills — stat_requirements stored as JSON object (e.g. {"physique": 3, "reflexes": 2}). is_magic bool gates external-only unlock.
- conditions — resistance_type maps to Physical / Affliction / Psychic. target specifies which body parts it can affect.
- enemies — category is one of Mob / Elite / Boss / Super Boss. resistances stored as JSON object.
- enemy_phases — behavior stored as JSON describing action patterns, movement rules, and special triggers per phase.
- NPC table is scaffolded now but content is populated later. base_inclinations stores tag IDs the NPC responds to positively or negatively.

4. Data Flow Diagram

Shows how data moves through the game at runtime across four layers: player input, logic engines, storage, and presentation.

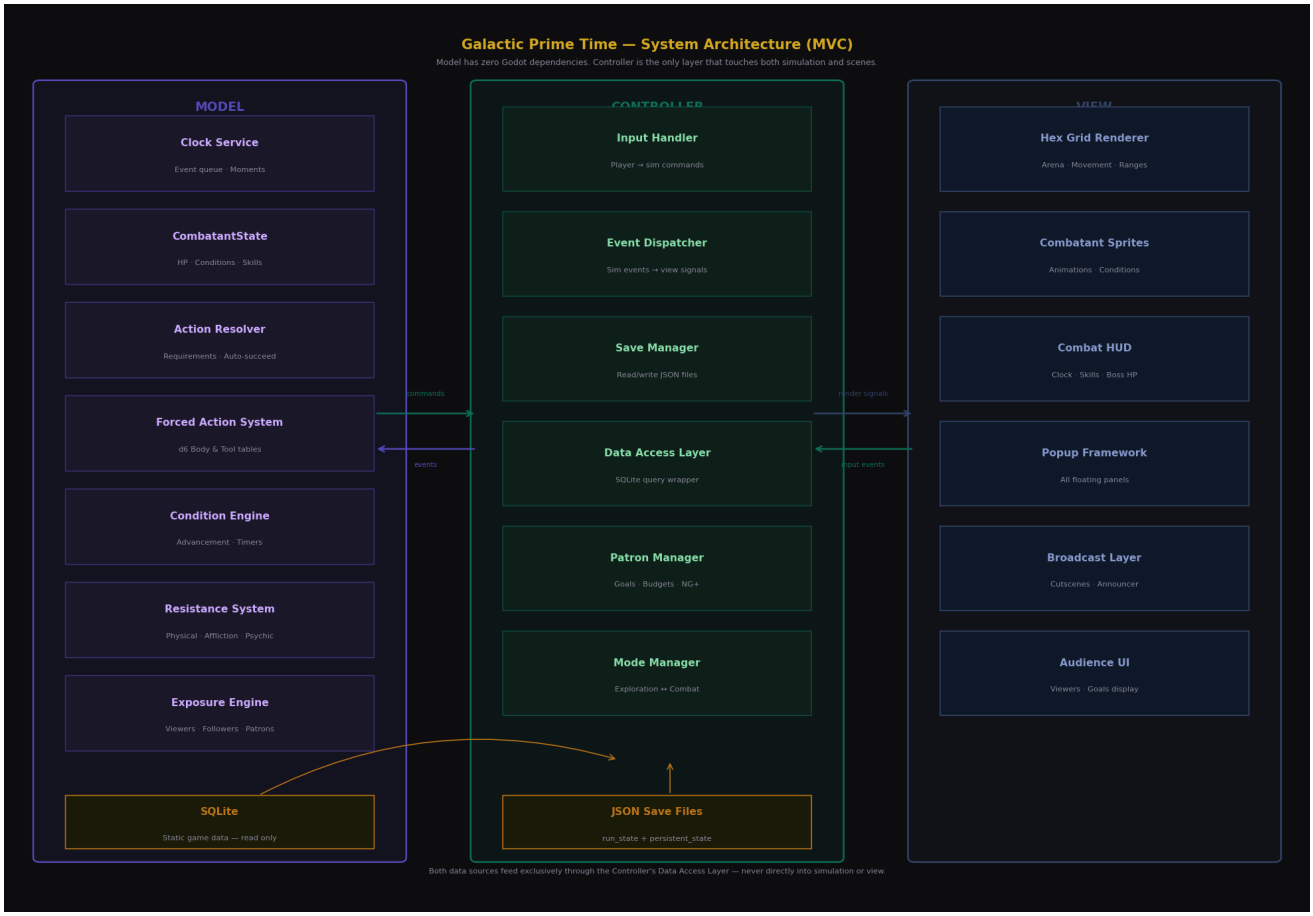
Data Flow Diagram — runtime data movement across all four layers

Key flows to understand:

- Player input enters through the Controller's Input Handler — never directly into simulation.
- Simulation emits events. Controller routes those events to the correct scene nodes as signals.
- SQLite and JSON files both feed exclusively through the Controller's Data Access Layer and Save Manager.
- On checkpoint: run_state.json is serialized to a checkpoint snapshot file. On death: snapshot is restored.
- Persistent state (races, patrons, NG+) is never wiped — it survives all deaths and run endings.

5. System Architecture Diagram

Internal component breakdown within each MVC layer, showing what each contains and how layers communicate.



System Architecture — MVC component breakdown with data source connections

Communication rules:

- Model → Controller: simulation emits events (action resolved, condition advanced, combatant died, etc.)
- Controller → Model: sends commands (declare action, advance clock, apply condition, etc.)
- Controller → View: emits Godot signals that scenes subscribe to.
- View → Controller: UI events (button pressed, hex clicked, popup opened) sent as method calls.
- SQLite → DAL only. JSON saves → SaveManager only. Neither the simulation nor scenes touch storage directly.

6. Combat System Rules

The Clock

Combat runs on a shared 10-Moment Clock counting downward. All combatants act on the same timeline — actions are declared, costs tallied, resolution overlaps. No turns. When Clock reaches 0 it resets and conditions advance.

Action Resolution

- Actions auto-succeed when stat requirements are met. No to-hit rolls.
- If requirements are NOT met, action still executes but triggers Forced Action of the appropriate type.
- Forced Action resolves AFTER the action completes — never before.
- Multi-Moment actions leave the character Exposed for their full duration.
- Exposed characters can have their Head targeted and may suffer lethal targeting.

Forced Action Tables (d6)

Body table	1=Tear Something (1 dmg to relevant part), 2=Lock-Up (part unusable 3M), 3=Condition Surge (advance one condition), 4=Drop Item, 5=Shock Spike (+1 Shock tier), 6=Stumble (Exposed until next Moment)
Tool table	1=Whiff (action fails to impact), 2=Overcommit (become Exposed), 3=Collateral (ally/env affected), 4=Slip (unarmed until next M), 5=Strained Grip (+1M cost next action), 6=Overextension (next action delayed 1M)

Localized HP

Head	2 HP · Lethal · Cannot be targeted unless enemy is Exposed, Helpless, or Overwhelmed
Torso	5 HP · Lethal · Always targetable
Arms (each)	2 HP · Non-lethal · Disabled at 0: drops item, two-handed impossible
Legs (each)	3 HP · Non-lethal · Disabled at 0: movement = Forced Action, becomes Exposed

Conditions

Bleeding · Crushed · Suffocation · Chilled · Exhausted · Infected · Burn · Poison · Dissolution. Each has tiers that advance per Clock reset. Conditions have three states: Active, Delayed, Resolved. Most in-combat treatments only Delay — full resolution requires downtime.

Resistances

Physical	Flat reduction to Bleed/Crush/Burn damage. Source: Reflexes stat cap — every 12 pts = 1 resistance of chosen type.
Affliction	Tier-based immunity to Chill/Poison/Infection. Source: surviving afflictions (unlocks passive skill per type).
Psychic	Tier-based immunity to Dissolution. Source: Mind stat cap — every 15 pts = 1 psychic resistance.

Shock Tiers

Tier 1 — Shout	Cry out, breaks stealth, alerts enemies

Tier 2 — Stutter	Freeze or seize, current action fails
Tier 3 — Faint	Collapse, unconscious 1 Clock, drop held item
Tier 4 — Helpless	Exposed for rest of combat

7. Key Systems

Skills

Start at level 0 (revealed, no effect). Default cap 5, raisable to 10 via Patron Tokens (1 token = +1 cap). Thresholds at every level from 5 onward: Upgrade (add effects) or Mutate (change purpose entirely). Magic skills require external unlock — loot box, achievement, or Wizard's Tower Lounge module.

Tags

Earned through behavior, fulfilled conditions, or completed Goals. States: Acquired → Reinforced → Fading → Lost (re-acquirable). Tags modify: loot bias, crowd response, NPC reputation gain/loss, Goal reward weights. Tags from previous runs become Goal modifiers when a character Ascends or wins.

Exposure

Viewers	Active watchers. Drive hype momentum and reward potential. Fluctuate based on combat events, conditions, and goals.
Followers	Recurring audience. Stabilize the reward floor. Grow from sustained Viewer presence.
Patrons	Paying supporters. Set Goals for rewards. Each new Patron awards 1 Patron Token.
Patron Tokens	Spent to raise a skill cap by 1 (max cap 10). Also earned by completing Directives. Boss Tokens convert at 3:1.
Camera Call	Charm stat cap mechanic. Every 20 Charm = 1 stack. Each stack doubles Exposure gains for a target for their current or next action.

NPC Reputation (Infrastructure Only)

Each NPC has a numeric reputation score per character stored in run_state.json. Tags act as multipliers on rep gain/loss. NPCs have base inclinations (tag IDs they respond to positively or negatively) stored in the NPC SQLite table. Reputation gates dialogue options and unlock unique branches regardless of score. Content TBD — infrastructure must be in place.

Ascension & NG+

Available after Floor 1. Two paths into the Patron pool:

- Corporate Ascension — retire mid-run via deal with the Corporation. Patron Goals reflect corporate interests.
- Winner's Arc — complete the show. Character returns home and becomes a galactic viewer. Goals reflect personal history.

Death is permanent — a character who dies cannot become a Patron. Patron budget is derived from Viewer/Follower/Patron count at time of Ascension or win. Tag history becomes Goal modifier weights. Winning permanently unlocks that character's race(s) as starter options.

Party Deployment

Unlimited roster. Deployment cap = battlefield hex count. Deploy at combat start or mid-combat for a Moment cost. Retrieval only when outside enemy attack range — units in range cannot be retrieved (or lose forces if a Stack). Exhaustion accumulates on deployed units, recovers on bench. Stacks (unnamed groups) use a simplified HP pool and collapse rather than enter conditions.

8. Campaign Structure & Content Schemas

All content is data, not code. When you are told to design a room, encounter, or dialogue sequence, you fill in these schemas. The engine loads and executes them.

Room Schema (KAN-43)

```
{ "id": "floor1_room_03c", "name": "Forest Clearing", "floor": 1, "route": "medium", "description": "A wide clearing split by a shallow ravine...", "available_actions": [ { "label": "Approach figures", "event_id": "evt_approach_npcs" }, { "label": "Circle the building", "event_id": "evt_circle_building" } ], "connections": [ { "direction": "north", "room_id": "floor1_room_02c" }, { "direction": "east", "room_id": "floor1_room_04c" } ], "encounter_ref": null, "npc_refs": ["npc_aldric", "npc_mira"], "environmental_lore": "Scorch marks on the northern wall suggest this isn't the first fire.", "checkpoint": false, "boss_room": false }
```

Encounter Schema (KAN-44)

```
{ "id": "enc_incineradile", "name": "Incineradile", "arena_hex_count": 41, "enemy_list": [{ "enemy_id": "incineradile", "count": 1, "position_hint": "center" }], "phases": [ { "phase": 1, "trigger_condition": "start", "behavior": "patrol" }, { "phase": 2, "trigger_condition": "network_hp_below_30", "behavior": "aggressive" } ], "special_conditions": ["surface_immune", "breach_required"], "breach_condition": { "method": "bleed_t2_any_part_or_7plus_damage", "description": "Bleed T2 on any part OR 7+ damage in a single hit" }, "rewards": [{ "type": "boss_token", "ref": "bronze" }], "death_explosion": { "radius": 19, "instant_kill": true }, "cinematic_intro": "broadcast_incineradile_intro" }
```

Dialogue / Event Schema (KAN-45)

```
{ "id": "evt_approach_npcs", "type": "dialogue", "speaker": "npc_aldric", "nodes": [ { "id": "node_01", "text": "Stay back! We're handling this.", "conditions": [], "choices": [ { "label": "Who's in there?", "next_node_id": "node_02", "event_triggers": [] }, { "label": "Stand aside.", "next_node_id": "node_03", "event_triggers": ["rep_aldric_-10"] } ] } ] }
```

9. Godot Project Folder Structure

Set up this exact structure on project init (KAN-21). Every file must live in its correct folder — the architecture depends on it.

```
/ ■■■ simulation/ # Model — zero Godot deps ■ ■■■ clock.gd ■ ■■■ combatant_state.gd ■ ■■■ action_resolver.gd ■ ■■■ forced_action.gd ■ ■■■ condition_engine.gd ■ ■■■ resistance_system.gd ■ ■■■ exposure_engine.gd ■ ■■■ controller/ # Controller layer ■ ■■■ game_controller.gd # Autoload singleton ■ ■■■ save_manager.gd ■ ■■■ data_access_layer.gd ■ ■■■ scenes/ # View — pure presentation ■ ■■■ hex_grid/ ■ ■■■ combat/ ■ ■■■ exploration/ ■ ■■■ ui/ ■ ■■■ hud/ ■ ■■■ popups/ ■ ■■■ broadcast/ ■ ■■■ data/ ■ ■■■ seeds/ # Source of truth for SQLite ■ ■ ■■■ races.json ■ ■ ■■■ skills.json ■ ■ ■■■ conditions.json ■ ■ ■■■ items.json ■ ■ ■■■ enemies.json ■ ■■■ content/ # Room, encounter, dialogue ■ ■ ■■■ rooms/ ■ ■ ■■■ encounters/ ■ ■ ■■■ dialogue/ ■ ■■■ schemas/ # JSON schema definitions ■ ■■■ room.schema.json ■ ■■■ encounter.schema.json ■ ■■■ dialogue.schema.json ■ ■■■ saves/ # Runtime only — gitignore this ■ ■■■ run_state.json ■ ■■■ checkpoint.json ■ ■■■ persistent_state.json ■ ■■■ assets/
```

10. Jira Ticket Order — Full Build Sequence

Work tickets strictly in order. Do not begin a ticket until all tickets it depends on are marked Done in Jira.

v0.1 — Data Foundation

Ticket	Summary	Depends On
KAN-8	Design SQLite schema — all static tables	—
KAN-9	Write SQLite migration scripts	KAN-8
KAN-10	Create static seed data files	KAN-9
KAN-11	Build first-launch seed pipeline	KAN-10
KAN-12	Build data access layer (DAL)	KAN-11
KAN-13	Define JSON save file schemas	KAN-11

v0.2 — Core Combat Engine

Ticket	Summary	Depends On
KAN-14	Implement Clock event queue	KAN-12
KAN-15	Implement CombatantState class	KAN-12
KAN-16	Implement action resolver	KAN-14, KAN-15
KAN-17	Implement Forced Action system (Body & Tool tables)	KAN-16
KAN-18	Implement condition advancement engine	KAN-16
KAN-19	Implement resistance system	KAN-18
KAN-20	Write headless combat simulation unit tests	KAN-17, KAN-19
KAN-44	Define encounter content schema	KAN-16

v0.3 — Godot Scaffolding

Ticket	Summary	Depends On
KAN-21	Godot project setup & folder structure	KAN-20
KAN-22	Build GameController singleton (MVC controller layer)	KAN-21
KAN-23	Build hex grid renderer (exploration & combat zoom)	KAN-21
KAN-24	Build SaveManager (checkpoint snapshot & restore)	KAN-22, KAN-13
KAN-45	Define dialogue & event schema	KAN-22

v0.4 — Character & Party

Ticket	Summary	Depends On
KAN-25	Build character creation system	KAN-22
KAN-26	Implement localized HP system	KAN-25

KAN-27	Implement skill system runtime	KAN-25
KAN-28	Implement inventory system	KAN-27
KAN-29	Implement party deployment & bench system	KAN-26

v0.5 — Map & Exploration

Ticket	Summary	Depends On
KAN-30	Build room state machine	KAN-29
KAN-31	Build Floor 1 map data (rooms, corridors, routes)	KAN-30
KAN-32	Implement exploration mode controller	KAN-31
KAN-33	Implement checkpoint & rewind system	KAN-32
KAN-43	Define room content schema	KAN-31

v0.6 — UI Layer

Ticket	Summary	Depends On
KAN-34	Build base HUD scene (broadcast frame + ticker bar)	KAN-33
KAN-35	Build reusable popup framework	KAN-34
KAN-36	Build all popup content scenes	KAN-35
KAN-37	Build combat HUD elements (clock bar, timeline, skills)	KAN-35

v0.7 — Progression & Audience

Ticket	Summary	Depends On
KAN-38	Implement Exposure engine (Viewers, Followers, Patrons)	KAN-36
KAN-39	Implement Tag system runtime	KAN-38
KAN-40	Implement Goal & Directive system	KAN-39
KAN-41	Implement Ascension system	KAN-40
KAN-42	Implement NG+ persistent state & race unlock system	KAN-41

11. Codex System Prompt & Per-Ticket Instructions

Paste the following block as your system prompt at the start of every new Codex or ChatGPT session. Do not modify it between tickets — the architecture rules must be consistent across all sessions.

■■ *SYSTEM PROMPT (copy everything below this line)* ■■■■■■■■■■■■■■■■■■■■■■■■■■■■

```
You are a senior game developer working on Galactic Prime Time, a 2.5D tactical RPG built in Godot 4 using GDScript. The game is a reality TV show where abducted humans compete in a dungeon crawl broadcast live to an alien audience. ARCHITECTURE — strict MVC, never violate these rules: - Model: headless simulation classes in /simulation/. Zero Godot dependencies. Fully serializable to JSON. - Controller: GameController autoload singleton in /controller/. Owns SaveManager and DataAccessLayer. This is the ONLY layer that touches both simulation and scenes. - View: Godot scene tree in /scenes/. Pure presentation. Listens to GameController signals only. Never queries SQLite directly. Never reads simulation state directly. DATA: - Static game data (skills, items, races, conditions, enemies, tags, NPCs) lives in SQLite. Seeded on first launch from /data/seeds/ JSON files. Read-only at runtime. - ALL SQLite queries go through DataAccessLayer exclusively. No exceptions. - Dynamic run state serialized to /saves/run_state.json (wiped on run end, restored on death). - Persistent state (unlocked races, patron pool, NG+ flags) in /saves/persistent_state.json (never wiped). - Content (rooms, encounters, dialogue) stored as JSON files in /data/content/ following defined schemas. COMBAT RULES: - Shared 10-Moment Clock counting downward. No turns. Actions overlap on the same timeline. - Actions auto-succeed when stat requirements are met. No to-hit rolls ever. - Failed requirements trigger Forced Action (d6 Body or Tool table), resolved AFTER the action. - Multi-Moment actions leave the character Exposed for their full duration. - Localized HP: Head 2HP (lethal), Torso 5HP (lethal), Arms 2HP each, Legs 3HP each. - Conditions advance each Clock reset. Multiple lethal timers can run simultaneously. - Resistances: Physical (flat, from Reflexes cap), Affliction (tier, from skill), Psychic (tier, from Mind cap). CURRENT TASK: [INSERT TICKET NUMBER AND SUMMARY HERE] Before writing any code: confirm you understand the architecture and ask clarifying questions if needed.
```

Per-Ticket Workflow

1. Open a new Codex session.
2. Paste the system prompt above, replacing [INSERT TICKET NUMBER AND SUMMARY HERE] with the current ticket.
3. Paste the full ticket description from Jira as the first user message.
4. Ask Codex to output files to the correct folder paths as defined in Section 9.
5. After the session, mark the ticket Done in Jira before starting the next.
6. If Codex produces something that contradicts this document, this document wins.

Example first message (KAN-8):

```
We are working on KAN-8: Design SQLite schema — all static tables. Define all tables from the handoff document: races, characters, skills, skill_thresholds, character_skills, tags, character_tags, items, modifiers, item_modifiers, inventory, conditions, condition_tiers, enemies, enemy_phases, patron_goals, NPC. Output: 1. A migration SQL file saved to /data/migrations/001_initial_schema.sql 2. A brief explanation of any design decisions, especially around JSON columns.
```